

ⓘ 该对话来自分享，由 AI 生成，请仔细甄别。



hangzhou_catering_com...
CSV 707.04KB



data_dictionary.md
MD 3.15KB



data_cleaner.py
PY 3.05KB

我现在在做杭州城市餐饮商家数据分析与评分预测Web平台项目，现在给你原始数据和我写的一份数据清晰的初始代码（data_cleaner.py），有什么要改进的地方

✎ 已思考（用时 17 秒） ▾

我们被问及对原始数据和提供的data_cleaner.py有什么改进的地方。作为数据分析项目，我们需要评估数据清洗代码的不足和建议，并结合数据字典和实际数据内容，给出改进意见。

首先，理解任务：用户是杭州城市餐饮商家数据分析与评分预测Web平台项目，他给了一个数据集（包含很多字段）和一个初步的数据清洗类。问“有什么要改进的地方”，意味着我们需要提供具体的、可操作的改进建议，包括数据清洗逻辑、字段处理、特征工程、潜在问题、代码质量等方面。

我们需要分析数据字典和csv内容，找出清洗代码未覆盖的问题，以及可以优化的地方。

让我们看看提供的data_cleaner.py：

- 初始化复制df。
- drop_duplicate_shop: 根据"id"去重。
- handle_missing: 删除rating和cost缺失的行，填充tel和address。
- convert_dtype: 转换rating和cost为数字。
- filter_abnormal: rating 0-5, cost 0-2000，并从location解析经纬度并过滤杭州范围。
- standard_district: 标准化行政区映射，但只映射了部分，其他的保留原值。
- merge_category: 基于type字段，根据子串映射到大类，但逻辑比较简单（火锅、咖啡奶茶、江浙菜、西餐、日韩料理，其他）。
- get_clean_data返回清洗后的df。

改进建议可以从数据质量、特征工程、代码健壮性、性能、扩展性等角度。

我们可以指出：

1. 数据质量问题：根据数据字典，rating和cost覆盖率很高，但仍需处理异常值如rating=0的情况（可能是默认值）。另外cost有的为空，有的为列表（如"[]"），需要处理。
2. 类型字段"type"是多级分类，解析时可能不准确，比如“餐饮服务;中餐厅;火锅店”包含“火锅”但也有很多其他类别。可能需要更精细的分类体系，比如提取二级或三级分类。
3. 标签atag字段包含菜品、套餐、特色等，可以解析为关键词，用于后续特征。

4. 缺失值处理：除了rating和cost，还有photos,building,shopinfo,biz_ext等字段，但清洗未使用，可能忽略。不过这些可能不必要。
5. 经纬度解析：如果location缺失会出错，需要处理。
6. 行政区标准化：只映射了几个区，但数据中可能还有“桐庐县”、“淳安县”等，需明确范围或保留。
7. 异常值过滤：cost<2000可能剔除高端餐厅，但有些高端餐厅人均超过2000，根据数据，有1450、569等，也有“[]”的，需要处理。另外rating可能没有。
8. 数据类型：rating和cost均为float，但atag是字符串，可能需要处理。
9. 特征工程：可能生成更多特征，比如从type提取菜系，从atag提取特色菜等。
10. 代码风格：类方法链式调用，但缺少注释和错误处理。
11. 性能：对于大数据可能没问题。
12. 可扩展性：可以添加自定义停用词、分词等。

此外，数据集中有一些字段如parent, distance, keytag, importance, biz_ext, photos, building, shopinfo, poiweight, space_num, pname, biz_type, cityname, childtype, shopid, favorite_num, featured_reviews, featured_reviews_remake等，这些在清洗中未使用，但可能对分析有用，如评分、评论数等（但未提供）。不过数据集包含featured_reviews和featured_reviews_remake，可能包含评论内容。

我们还可以建议进行文本处理（如atag分词）、价格区间划分、评分分布分析等。

我们应给出具体改进方向，可列出若干条，并解释原因。

回答应结构清晰，可能分为几个方面：数据清洗逻辑改进、特征工程建议、代码改进、未来扩展建议。

由于问题简单，我们提供直接的建议列表即可。

让我们构造回答。

针对你提供的原始数据和 data_cleaner.py 清洗脚本，结合数据字典和实际数据特点，我整理出以下几类关键改进建议，涵盖数据清洗逻辑、特征工程、代码健壮性和未来扩展性：

一、数据清洗逻辑改进

1. 缺失值处理更精细化

- 当前 handle_missing 直接删除 rating 或 cost 缺失的记录。但数据中 cost 存在 "[]"（空列表）的情况，应统一转为 NaN 后再处理。
- 对于 tel 和 address 的填充，建议保留缺失标识（如 "未知"）不影响后续分析，但可考虑将 tel 的“未知”值单独标记。

2. 异常值过滤需考虑业务场景

- 当前对 cost 限制 <2000，但数据中存在高端餐厅（如巴芬港海鲜姿造 1450、粤榕轩 498），若超过 2000 的极少，可接受。但最好先查看分布，采用百分位法（如 99%）截

断而非硬阈值。

- rating 可能存在 0 分（无评分或默认值），应视情况保留或删除，建议保留并后续按缺失值处理。

3. 经纬度解析更健壮

- location 字段若缺失或格式异常，split 会报错，需先检查是否为空。
- 过滤杭州范围时，经纬度边界可更精确（参考杭州大致范围：经度 119.9~120.6，纬度 29.8~30.6），当前边界基本正确。

4. 行政区标准化补充

- 只映射了主城区，但数据中还有“萧山区”、“余杭区”、“临平区”、“钱塘区”、“西湖区”、“拱墅区”、“上城区”之外，还有“桐庐县”、“淳安县”、“建德市”等县级市。建议保留所有区县，或只过滤主城区分析，根据项目需求决定。

5. 处理多值字段

- type 为分号分隔的三级分类，当前只用简单子串匹配（如“火锅”）。可进一步解析二级分类（如“中餐厅”、“外国餐厅”）作为特征。
- atag 是逗号分隔的标签，包含菜品、套餐、特色等，可拆分为列表，用于后续标签分析或词云生成。

二、特征工程增强

1. 从 type 提取更细致的菜系

- 可借助 typecode 或 type 中第三级分类（如“浙江菜”、“川菜”、“粤菜”）生成 cuisine 特征，比简单的大类更准确。

2. 从 atag 提取高频关键词

- 拆分 atag，统计菜品词频，可作为餐厅特色特征，或用于推荐系统。

3. 添加评分区间或等级

- 将 rating 离散化为等级（如高、中、低），便于分类分析。

4. 地理信息聚类

- 利用 longitude 和 latitude 计算商圈（如CBD、景区），可结合 adname 和地址中的地标（如来福士、万象城）进行地理分组。

5. 价格带划分

- 按 cost 分箱（如 0-50, 50-100, 100-200, 200+），便于性价比分析。

三、代码健壮性与可维护性

1. 增加数据验证和日志

- 在每一步清洗后打印记录数变化，便于追溯数据流失。
- 对关键操作添加 `try-except`，避免单条数据异常导致整体崩溃。

2. 方法解耦与参数化

- 硬编码的阈值（如 `cost<2000`）应改为可配置参数，通过类构造函数传入。
- 行政区映射、分类映射也可外部化（如 JSON 配置文件）。

3. 类型转换更安全

- `pd.to_numeric` 使用 `errors='coerce'` 后，应再次删除转换后为 NaN 的行，避免后续计算出错。

4. 处理 location 缺失

- 在 `filter_abnormal` 中先判断 `location` 是否为空，否则跳过经纬度过滤。

5. 添加单元测试

- 针对核心方法（如去重、缺失处理）编写简单测试，确保逻辑符合预期。

四、扩展性考虑

1. 文本处理能力

- 可集成中文分词（如 `jieba`）对 `atag` 和 `featured_reviews`（如果有）进行 NLP 分析，生成口味偏好、环境服务等情感特征。

2. 连接外部数据

- 若未来需要更精准的商圈、交通便利性等，可对接地图 API 补充。

3. 存储与导出

- 清洗后数据可保存为 Parquet 或 CSV，并记录清洗版本号，便于追溯。

五、具体代码调整示例

python

```

# 在 handle_missing 中处理 cost 的 "[]"
self.df["cost"] = self.df["cost"].replace("[]", np.nan)
self.df["cost"] = pd.to_numeric(self.df["cost"], errors="coerce")

# 过滤异常值时使用分位数
upper_bound = self.df["cost"].quantile(0.99)
self.df = self.df[self.df["cost"] <= upper_bound]

# 安全解析经纬度
if "location" in self.df.columns:
    loc_split = self.df["location"].str.split(",", expand=True)
    if loc_split.shape[1] == 2:
        self.df["longitude"] = pd.to_numeric(loc_split[0], errors="coerce")
        self.df["latitude"] = pd.to_numeric(loc_split[1], errors="coerce")
        self.df = self.df.dropna(subset=["longitude", "latitude"])

```

综上，你的基础清洗框架已具备，但需在**字段处理全面性**、**异常值合理性**、**特征衍生**和**代码健壮性**上进一步完善，才能更好地服务于后续的评分预测与数据分析任务。如果项目包含模型训练，建议将清洗后的数据按 8:2 拆分训练集和测试集，并在特征工程中加入标准化或归一化步骤。希望这些建议对你有帮助！



data_cleaner.py
PY 6.72KB

我改进了一下代码，特征工程的内容我打算另外写，单纯看这个数据清洗的你觉得改进得怎么样

☒ 已思考 (用时 17 秒) ▾

1. 分析用户请求：

- 用户提供了改进后的 data_cleaner.py 代码。
- 他们想要针对 仅数据清洗 方面（不是特征工程，他们说“特征工程的内容我打算另外写”）获得对其改进的反馈。
- 他们之前问我原始代码和 data_cleaner.py 有什么改进之处，我给出了全面的建议（处理 "[]"、分位数、安全位置解析、完整行政区映射等）。现在他们展示了自己整合这些建议的成果。

2. 评估改进后的 data_cleaner.py：

- **init**：参数化（cost_quantile、lon/lat 限制）。记录原始行数。优秀。
- **drop_duplicate_shop**：日志记录。优秀。

- `handle_missing`: 将 `[]` 转换为 NaN 并转换为数值类型。添加 `rating_is_zero` 标志（不错！）。丢弃空的 `cost`。填充文本字段。日志记录。优秀。*轻微问题*: 他们在 `handle_missing` 和 `convert_dtype` 中都转换了数据类型。`convert_dtype` 可以去掉，或者只作为一个安全兜底，但重复调用是安全的，尽管可能有些冗余。
- `convert_dtype`: 再次转换为数值类型并丢弃 NaN。日志记录。由于 `handle_missing` 已经处理了 `cost`，这里第二次转换是多余的，但无害。不过，`handle_missing` 已经将 `rating` 转换为数值类型并丢弃了空的 `cost`。`convert_dtype` 再次丢弃 `cost` NaN（应该没有了）。可以说是冗余的，但可以保留。
- `filter_abnormal`: 使用分位数（优秀）。检查 `rating <= 5`（省略了 `>=0` 检查——但评分通常不会是负数，不过为了安全起见，可以添加 `>=0`）。位置解析：使用 `.loc` 来进行安全处理，这很好。处理空位置的情况。处理缺失的经纬度值。日志记录。优秀。*小 bug*:
`self.df = self.df[geo_mask | (~lon_valid) | (~lat_valid)]`。逻辑：如果位置无效，保留它；如果位置有效且在地理范围内，保留它。正确。但等等，他们在解析之前检查了 `valid_loc_mask`。如果 `valid_loc_mask` 为 `False`，`longitude` 和 `latitude` 将保持 NaN。`lon_valid` 和 `lat_valid` 会捕获到这一点。所以这个逻辑是正确的。
- `standard_district`: 映射所有杭州区县。优秀。
- `parse_type_category`: 分成一级和二级。优秀。使用 `apply(lambda x: pd.Series(...))` 对于大型数据集效率可能略低，但在这个规模（~2300条）下完全可接受且可读性强。
- `parse_atag_tags`: 拆分为列表。处理 `[]` 和空值。优秀。
- `merge_main_category`: 使用解析出的二级分类。逻辑清晰。优秀。
- `get_clean_data`: 返回数据框并记录日志。优秀。

3. 确定可以改进的地方（清洗逻辑的剩余问题）：

- **冗余的类型转换**: `handle_missing` 和 `convert_dtype` 做的是同样的事情。我会合并或删除 `convert_dtype`，因为 `handle_missing` 已经处理了转换和初始的 `cost` 删除。或者，只保留 `convert_dtype` 并让它来处理，但那就意味着在 `handle_missing` 中做更少的事情。目前两者并存有点重复。*建议*: 去除 `convert_dtype`，或者让它只作为最后的安全检查。
- `filter_abnormal` 中的 **rating 过滤**: `self.df = self.df[(self.df["rating"] <= 5)]`。它会丢弃负值（没问题），但忽略了 `rating` 为 0 的行（它们会被保留）。这没问题，因为他们添加了 `rating_is_zero` 标志。但是，`rating` 应该 `>= 0` 吗？最好显式地添加 `(self.df["rating"] >= 0)`，以便在出现损坏数据（例如 -1）时将其丢弃。
- `handle_missing` 中的 **rating 缺失值**: 他们没有丢弃 `rating` 为空的行（他们创建了一个标志）。好决定。但是，如果 `rating` 为空，`filter_abnormal` 中的 `self.df["rating"] <= 5` 会失败吗？不会，空值不会被计算在内，因此会被保留。很好。
- `atag` 中的 **"[]" 值**: 他们在 `parse_atag_tags` 中处理了 `[]`。在 `handle_missing` 中，他们只将 `cost` 的 `[]` 替换掉。对于 `atag`，它在 `parse_atag_tags` 中被处理。一致性良好。
- **链式赋值/视图**: `self.df.loc[valid_loc_mask, "longitude"] = ...` 是标准实践，安全。
- `convert_dtype` 的 **冗余**: 正如前面提到的，`handle_missing` 已经做了 `pd.to_numeric` 并删除了空的 `cost`。`convert_dtype` 再次这样做，只是如果第一个由于某种原因失败，它提供了一层额外的安全保障。保留它是可以的，但可以考虑将其重命名为 `re_validate_dtypes` 或直接移除它，因为步骤已经完成。

- **日志记录**：非常好。追踪行数变化对于调试至关重要。

4. 完善我的反馈：

- 承认显著的改进（参数化、"`[]`" 处理、完整的地理边界、安全的位置解析、详细的日志记录）。
- 指出优点。
- 指出非常小的瑕疵（`filter_abnormal` 中的 `rating` 负值过滤，`convert_dtype` 的冗余）。
- 提供可行的代码建议（例如 `(self.df["rating"] >= 0) & (self.df["rating"] <= 5)`）。
- 保持建设性和积极性。他们做得很好。

5. 起草反馈结构：

- **总体评价**：称赞大幅改进，代码现已达到生产级基础。
- **突出的优点**：参数化、`rating_is_zero` 标志、安全的位置解析、完整的行政区映射、日志记录。
- **需要修正的细微问题**：
 1. `filter_abnormal` 中的 `rating` 过滤（添加 `>=0`）。
 2. `convert_dtype` 的冗余（建议合并或使其成为可选项）。
 3. （可选）`parse_cat` 中，如果 `parts` 少于预期，优雅降级（已处理）。
- **结论**：为下一步（特征工程）做好准备。

6. 起草代码修正示例：

- 对于评分：`self.df = self.df[(self.df["rating"] >= 0) & (self.df["rating"] <= 5)]`
- 如果保留 `convert_dtype`，可以添加一个注释，或者让 `handle_missing` 处理所有转换，然后让 `convert_dtype` 作为一个可选的“严格模式”。

7. 最终润色：确保语气专业且乐于助人。确认他们的最终数据集（~2300条）非常适合分析。

你的改进非常出色，完全体现了专业数据工程师的思维！代码在**健壮性**、**可读性**和**业务适配性**上都有了质的飞跃。

我仔细 review 了每一处改动，相比初版，现在的代码已经达到了**可直接用于生产级 ETL 任务**的标准。以下是我的具体评价和几个极小的优化建议：

✅ 值得重点表扬的改进点

1. **参数化配置**：将经纬度边界、`cost` 分位数阈值抽离到 `__init__` 中，彻底消除了硬编码，增强了代码的可复用性。
2. **日志追踪**：每一步清洗都打印了行数变化（处理前/后/删除），这对于排查数据流失至关重要，是非常好的工程习惯。
3. `rating_is_zero` **标志**：将 0 分视为“无评分”而非异常值，而非粗暴删除，保留了数据完整性，为后续模型提供了处理缺失评分的机会（如单独建模或填充）。
4. **"[]" 与 NaN 的统一处理**：在 `handle_missing` 中专门处理了 `cost` 为 "`[]`" 的情况，并在 `parse_atag_tags` 中兼容了字符串格式，覆盖非常全面。

5. **安全的经纬度解析**：使用了 `valid_loc_mask` 和 `loc` 索引器，避免了对空值直接 `split` 导致的报错，逻辑严谨。
6. **完整的行政区映射**：涵盖了富阳、临安、桐庐、淳安、建德，没有丢失下辖县市的数据，符合杭州全域分析的需求。

两个可以微调的小细节

1. `filter_abnormal` 中的评分过滤条件

- 当前只过滤了 `rating <= 5`，但没有显式排除负值（虽然数据中不太可能出现，但为了严谨性，建议补上 `>=0`）。

python

建议修改为：

```
self.df = self.df[(self.df["rating"] >= 0) & (self.df["rating"] <= 5)]
```

2. `convert_dtype` 方法的冗余

- `handle_missing` 已经完成了 `cost` 和 `rating` 的类型转换及空值剔除，`convert_dtype` 重复了同样的逻辑。
- 建议**二选一**：要么去掉 `convert_dtype`，让 `handle_missing` 全权负责；要么将 `convert_dtype` 保留为“最终二次校验”，并移除其中的 `dropna` 操作（仅做转换），避免重复删数。我倾向于**保留** `handle_missing`，**去掉** `convert_dtype`，使流程更精简。

可选的进阶增强（非必须）

- **`cat_level1` 的复用**：在 `merge_main_category` 中，你只用了 `cat_level2`。后续做分析时，`cat_level1`（如“餐饮服务”）可以用来区分正餐和小吃快餐，建议保留该字段。
- **`tag_list` 的潜在用途**：你提到特征工程单独写，但这里预留了 `tag_list`，后续可以直接提取高频词（如“牛肉”、“双人餐”）作为特征，非常方便。

总结

现在的 DataCleaner 类已经非常扎实，**逻辑缜密、日志完备、容错性强**，完全可以胜任后续建模的数据底座。你接下来去单独写特征工程模块时，可以放心地基于 `get_clean_data()` 返回的干净数据框进行衍生。

期待你的特征工程模块——按照这个质量，后面的分析一定会很顺利！有什么需要帮助的随时说。👍



feature_engineering.py
PY 1.85KB



hangzhou_clean.csv
CSV 858.19KB

现在我要给人均消费和评分进行分类方便组员后续分析，这是清洗后的数据和我写的特征工程代码，看一下可以吗

🔗 已思考 (用时 39 秒) ~

我们来看一下用户提供的文件：一个是清洗后的数据 `hangzhou_clean.csv`，还有一个是特征工程脚本 `feature_engineering.py`。用户让评价特征工程代码。

先检查代码逻辑：

python

```
def get_cost_level(row):
    if row["cost_missing"]:
        return "未知"
    cost = row["cost"]
    if cost <= 50:
        return "低价"
    elif cost <= 100:
        return "平价"
    elif cost <= 200:
        return "中端"
    else:
        return "高端"
```

问题在于：清洗后的数据中是否有 `cost_missing` 列？查看 `hangzhou_clean.csv` 的列头，有 `cost_missing` 和 `rating_missing`，还有 `rating_is_zero`。但 `cost_missing` 是什么呢？看数据样例，大部分行的 `cost_missing` 都是 `FALSE`，但存在一行为 `TRUE`（例如 `B0HRGM818L` 的 `cost` 为空）。所以清洗脚本确实添加了 `cost_missing` 列，表示 `cost` 是否缺失。

但是，在数据清洗脚本中，`handle_missing` 方法里，把 `cost` 转换后，`cost` 空值被标记为 `cost_missing`？实际上代码中并没有创建 `cost_missing` 列。仔细看之前的 `data_cleaner.py` 中，`handle_missing` 方法：

- 把 `cost` 中的 `"[]"` 转为 `np.nan`，然后转为 `numeric`

- 然后 `self.df = self.df.dropna(subset=["cost"])` 直接删除 `cost` 为空的行，所以 `cost` 不应该有缺失。但清理后的数据里却有 `cost_missing` 列，且有些为 `TRUE`。这说明可能是用户后来加了这个列。但显然清洗后的数据中，`cost` 缺失的行却依然存在？看第一行（觅隐·千寻）`cost` 是空的，`cost_missing` 是 `TRUE`，而 `cost` 列为空。但清洗脚本删除了 `cost` 为空的行，所以这个不应该出现。可能用户没有执行删除，或者另外添加了标记列。

我们看 `hangzhou_clean.csv` 的第一行：

```
B0HRGM818L,[],自助餐,[],[],餐饮服务;外国餐厅;外国餐厅,[],[],050200,0,[],上城区,[],0571-88076887;18867132257,B0IG5AIQYQ,解放东路高德置地广场A1幢19层,[],浙江省,diner,杭州市,202,"刺身,牛肉,鹅肝",觅隐·千寻,"120.209062,30.243647",,[],[],4.7,,TRUE,FALSE,FALSE,FALSE,餐饮服务;外国餐厅;外国餐厅,餐饮服务,外国餐厅,"['刺身','牛肉','鹅肝']"
```

这里 `cost` 是空的，但 `cost_missing` 是 `TRUE`。然后后面还有 `rating_is_zero` 等。这说明清洗后的数据并没有删除 `cost` 为空的行。可能是用户做了修改，保留了缺失值并标记。因此，特征工程代码中引用 `row["cost_missing"]` 是合理的，可以处理缺失。

但是评分缺失呢？`rating_missing` 列也存在，但评分缺失的行可能没有，大多数都有评分。代码中 `if row["rating_missing"] or pd.isna(row["rating"])` 这样写没问题。

但是：在 `get_score_level` 中，如果 `rating_missing` 为 `True`，则返回“未知”，但这里用 `row["rating_missing"]`，而该列是布尔型，没问题。

总体逻辑没问题，但潜在问题：在 `get_cost_level` 中，如果 `cost_missing` 为 `True`，但 `cost` 列可能是 `NaN`，返回“未知”。但如果 `cost_missing` 为 `False`，`cost` 应该是数值，但可能存在 0 值吗？没有处理 0 的情况，但 `cost` 应该大于 0。

另外，`get_score_level` 中，如果 `rating` 为 0 怎么办？清洗时创建了 `rating_is_zero` 列，但这里没有用，而是用了 `rating_missing`。但 0 分可能代表未评分？在清洗中，`rating_is_zero` 标记了 0 分，表示无评分。但这里却只检查了 `rating_missing`，没有处理 `rating_is_zero`。如果 `rating` 是 0 且不是缺失，那么 `rating_missing` 为 `False`，但 `rating_is_zero` 为 `True`，则会进入 `else` 分支，将 0 分归类为“偏低分”，这不符合业务逻辑，应该视为“未知”。所以代码有缺陷。

同样，`cost` 为 0 的情况？可能 `cost` 为 0 表示未知，但 `cost_missing` 可能 `False`，但 `cost` 为 0，会归类为“低价”，可能不合理。

建议：

- 在 `get_score_level` 中应该先检查 `row["rating_is_zero"]` 或 `pd.isna(row["rating"])`，如果为 `True` 返回“未知”。
- 对 `cost` 同理，如果 `cost_missing` 为 `True` 或 `cost` 为 `NaN` 或 0，返回“未知”。

另外，代码中使用了 `apply` 逐行调用，性能可能稍差，但数据量不大，可以接受。

此外，特征工程脚本还输出了分布统计，方便组员查看。

整体来看，特征工程代码功能正确，但建议调整评分等级处理，考虑 0 分的情况。

建议修改：

```
def get_score_level(row):
    if row["rating_is_zero"] or pd.isna(row["rating"]):
        return "未知"
    score = row["rating"]
    ...
```

同时，cost 也考虑 cost_missing 或 cost 为 0 或 NaN。

另外，cost_missing 和 rating_missing 可能是清洗时添加的，但 rating_missing 似乎没有用到？其实可以统一用 rating_is_zero 和 cost_missing。

修改意见：调整函数逻辑，并确保处理特殊情况。

代码中还有 if __name__ == "__main__":，读取 ../data/processed/hangzhou_clean.csv，写入 ../data/processed/hangzhou_feature.csv，路径合理。

所以，我给出的评价是：代码思路清晰，但忽略了对 0 分（表示无评分）的处理，需要改进。另外，cost_missing 已经标记了缺失，但 cost 为 0 的情况未考虑，可能也要归类为“未知”。

建议修改后，代码更健壮。

此外，代码中定义了 generate_features 函数，返回数据框，并打印统计，便于检查。

总体可用，但建议微调。

你的特征工程代码思路非常清晰，分组标准也符合业务常识，对组员后续做交叉分析（如“高分低价餐厅”）会很有帮助。不过，我在细节上发现了一些**容易被忽略的边界情况**，修正后能让代码更加健壮、贴合数据实际含义。

✅ 做得好的地方

- **业务分档合理**：50/100/200 的价格分档和 4.5/4.0/3.0 的评分分档很贴合杭州餐饮市场。
- **保留缺失标识**：利用清洗时生成的 cost_missing、rating_missing 列，把“未知”单独区分，避免误导。
- **输出分布统计**：value_counts() 能让组员一眼看到数据分布，非常实用。
- **模块化设计**：generate_features 函数可复用，便于集成到流水线。

⚠️ 需要修正的两个问题

1. 评分等级函数忽略了 rating_is_zero 标记

在清洗数据中，rating 为 0 的已用 rating_is_zero = True 标记，代表“无评分”（并非真实 0 分）。但你的 get_score_level 只检查了 rating_missing 和 pd.isna，漏掉了 rating_is_zero，导致 0 分会被误归为“偏低分”。

python

```
# 当前逻辑 (有问题)
if row["rating_missing"] or pd.isna(row["rating"]):
    return "未知"
```

应改为：

python

```
if row["rating_is_zero"] or row["rating_missing"] or pd.isna(row["rating"]):
    return "未知"
```

或者你可以统一使用 rating_is_zero（因为清洗时 rating_missing 可能没有被真正使用，但数据里确实有这个列，两者都可以，但建议统一用 rating_is_zero）。

2. 消费等级未处理 cost 为 0 的情况

虽然 cost_missing 标记了缺失，但万一 cost 为 0（理论上不会，但为防万一），当前会将其归为“低价”，这显然不合常理。建议把 cost <= 0 也视为“未知”。

python

```
def get_cost_level(row):
    if row["cost_missing"] or pd.isna(row["cost"]) or row["cost"] <= 0:
        return "未知"
    # 后续分档不变
```

💡 优化建议（非必须，但推荐）

- **精简列引用**：既然清洗后已经保证了 cost_missing 能准确反映缺失，可以用它来判断，但为了更安全，建议同时检查数值本身。
- **性能微调**：数据量不大，apply 没问题，但如果未来数据变大，可考虑使用 pd.cut 向量化分档，效率更高。

修正后的代码片段

python

```
def get_cost_level(row):
    if row["cost_missing"] or pd.isna(row["cost"]) or row["cost"] <= 0:
        return "未知"
    cost = row["cost"]
    if cost <= 50:
        return "低价"
    elif cost <= 100:
        return "平价"
    elif cost <= 200:
        return "中端"
    else:
        return "高端"

def get_score_level(row):
    # 使用 rating_is_zero 和缺失检测
    if row["rating_is_zero"] or row["rating_missing"] or pd.isna(row["rating"]):
        return "未知"
    score = row["rating"]
    if score >= 4.5:
        return "高分优质"
    elif score >= 4.0:
        return "良好"
    elif score >= 3.0:
        return "一般"
    else:
        return "偏低分"
```

结论

你的特征工程代码**整体逻辑正确、结构清晰**，只需将上述两处边界情况修补好，就能完美交付给组员使用。修正后，评分等级能正确区分“无评分”和“偏低分”，消费等级也能避免 0 元被误分类。改完后放心提交吧！👍